

Fast Parallel Image Rotation Algorithm

Dhrubajyoti Mandal
School of Computer Engineering
Kalinga Institute of Industrial Technology
Bhubaneswar, India
22053859@kiit.ac.in

Sourav Kumar Parida
School of Computer Engineering
Kalinga Institute of Industrial Technology
Bhubaneswar, India
22051032@kiit.ac.in

Subhadeep Bhadra
School of Computer Engineering
Kalinga Institute of Industrial Technology
Bhubaneswar, India
2205336@kiit.ac.in

Dr. Sujoy Dutta
Asst. Prof. & Asst. CoE
School of Computer Engineering
Kalinga Institute of Industrial Technology
Bhubaneswar, India
sdattafcs@kiit.ac.in

Chiranjeev Bhaya
Lead Engineer (Camera System)
Samsung R&D Institute
Bangalore, India
c.bhaya@samsung.com

Chandra Mohan V
Architect (Camera System)
Samsung R&D Institute
Bangalore, India
chandra.mv@samsung.com

Abstract—This paper presents a novel fast image rotation algorithm that leverages CPU parallelization while maintaining high image quality. The proposed method is a single-pass algorithm, derived from the existing Double-Line Rotation (DLR) method, which reduces computational complexity and generalizes the algorithm. Initially, a baseline for the given image is calculated to determine the starting line, which defines the initial point for each vertical or horizontal line in the image to be rotated. The corresponding pixels to be mapped are identified using floating point arithmetic, and trigonometric calculations are performed only once per line. This approach ensures precise image transformation with minimal computational overhead.
Index terms: Image rotation, line rotation image transform, double-line rotation (DLR), parallel image rotation.

I. INTRODUCTION

2-D Image Rotation has a number of applications in digital photography and image processing like image matching, alignment, [1] etc. Highly accurate image rotation is essential for certain image processing tasks such as feature extraction and matching. Whilst there are many state-of-the-art image rotation algorithms, they have limitations in terms of image quality and speed [2] [3]. Parallel algorithms for image processing aim to execute faster than sequential algorithms, however designing and scheduling a fast parallel image rotation is a challenge [4]. The most straightforward, and perhaps the most well known approach to implement image rotation is by using the rotation matrix [5], as described here

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} x - cx \\ y - cy \end{bmatrix} * \begin{bmatrix} \cos(\theta) & -\sin(\theta) \\ \sin(\theta) & \cos(\theta) \end{bmatrix}$$

where, θ is the angle of rotation, x' and y' are the new

coordinates of the rotated image, and (cx, cy) is the center of the original image, which is treated as the axis of rotation. Assuming the centre of rotation of the new image is (cx', cy') , the final new coordinates are:

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} x' + cx' \\ y' + cy' \end{bmatrix}$$

II. PREREQUISITES

A. Affine Transformation

Affine transformation is a fundamental technique in computer graphics and computational geometry that enables the manipulation of images through translation, scaling, rotation, shearing, and reflection. These transformations are mathematically represented by a combination of linear transformations and translations, preserving points, straight lines, and planes. The general form of an affine transformation in two dimensions is expressed as $y = Ax + b$, where A is a 2x2 matrix representing the linear transformation and b is the translation vector. For rotation, the matrix A is defined by trigonometric functions involving the rotation angle θ . The translation values t_x and t_y are calculated to center the rotated image within its new dimensions, ensuring that the image remains centered post-rotation. The transformation is particularly advantageous for its simplicity and ability to preserve the structure of the space.

Disadvantages: Despite its versatility, affine transformation can result in image distortions, particularly when the rotation angle is large or when combined with scaling or shearing. Additionally, it may introduce artifacts such as aliasing or loss of detail due to the discrete nature of digital images.

B. Image Rotation Using Hermite Expansion

The Hermite polynomial expansion is a more sophisticated method for image rotation, leveraging the orthogonality of Hermite polynomials to achieve smooth and accurate interpolation. Hermite polynomials are particularly effective in approximating functions, making them suitable for image processing tasks that require high precision. By expanding the image function using two-dimensional Hermite polynomials, the image can be rotated by transforming the coordinates with a rotation matrix. The rotated image is then reconstructed using the inverse Hermite expansion, ensuring smooth transitions and minimal distortion.

Disadvantages: The Hermite expansion method, while precise, is computationally intensive due to the need for evaluating and summing multiple polynomial terms. This complexity can make the algorithm slower compared to simpler methods like affine transformation. Additionally, the method may struggle with preserving fine details in images with high-frequency content.

C. Image Rotation Using Double Line Rotation

The Double Line Rotation (DLR) method divides the image into eight distinct zones, each governed by specific rotation equations and transformations. This method maps each pixel from the original image to two corresponding pixels in the target image, maintaining a consistent distance ratio between points throughout the rotation. This consistency is crucial for preserving both the intensity and geometric integrity of the pixels, making the DLR method particularly effective for applications that require feature extraction invariance. Each zone handles the image differently, applying horizontal and vertical line transformations, index manipulations, and logical transformations to achieve the desired rotation.

Disadvantages: The DLR method can be complex to implement due to the need to manage multiple zones and transformations, each with specific rules. This complexity can lead to inefficiencies, particularly when dealing with high-resolution images. Additionally, the method may introduce distortions at the boundaries between zones, leading to artifacts that can affect the visual quality of the rotated image.

III. PROPOSED METHOD

The proposed method is a single pass algorithm, which means that it performs the entire image rotation operation at once. The proposed algorithm predominantly affects pixels in the row major (width), or ones in the column major (height), depending on the angle of rotation, α . If $\alpha \in (\frac{\pi}{4}, \frac{3\pi}{4}] \cup (\frac{5\pi}{4}, \frac{7\pi}{4}]$, rotation is performed in the **column major**, and it is performed in the **row major** for the rest of the angles. Accordingly, for rotating the source image, entire row or column vectors of pixels are rotated as a single unit, instead of the more common approach, where each individual pixel is separately mapped. Due to this reason, it is necessary to accurately determine the initial coordinates from which the new row or column vector will begin. The coordinates of the rest of the pixels in the vector can then be determined by

simple floating point addition or subtraction, depending on the extent to which each of the said vectors are spread across the coordinate axes.

Apart from this, depending on α , the pixel processing order changes - i.e., whether the last row or column vector of pixels get processed first, or last. Since it has been found out that these values are very sensitive to α , the specific details of which are discussed in the later parts of this paper.

A. Rotation Of An Image In The Column Major

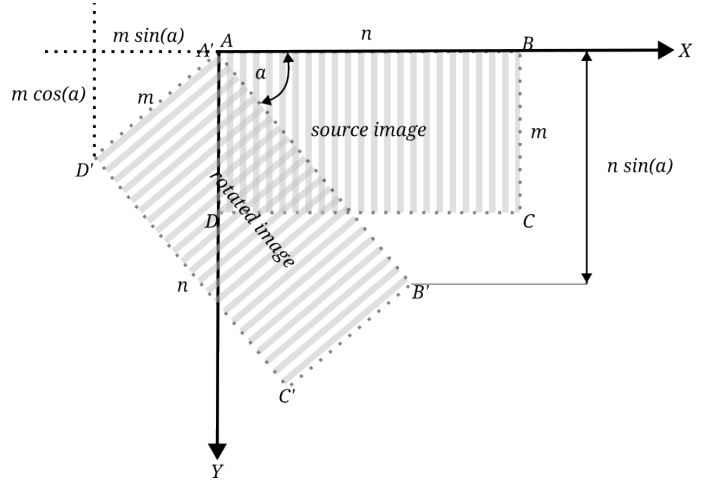


Fig. 1: Example demonstrating rotation of an image in the column major. Every column vector of pixels are rotated at once (Represented by the stripes in the image).

As seen in the example image in figure 1, for the given angle of rotation, column vectors were rotated (represented by the stripes). Each such vector to be rotated begins from the line $A'B'$. As such, the initial coordinates of each of these vectors can be determined by the equation of the same line, which is given by:

$$f(i) = \left\lfloor \frac{i}{\tan(\alpha)} \right\rfloor \forall \{i \in \mathbb{Z}, i \in [0, n \cdot \sin(\alpha))\} \quad (1)$$

Thus, the initial coordinates of the column vectors would be of the type $(f(i), i)$. Next, to determine the coordinates of the rest of the pixels in the column vector, as mentioned previously, we perform floating point addition or subtraction, depending on the extent to which each of the said vectors are spread across the coordinate axes.

In the example in figure 1, we can see that each column vector is spread through a length of $m \cdot \sin(\alpha)$ in the x -axis and through a length of $m \cdot \cos(\alpha)$ in the y -axis. It may also be observed that these values are the projections of any line perpendicular to line defined by the function in equation 1 on the coordinate axes.

We now define:

$$\delta x = |\sin(\alpha)|$$

$$\delta y = |\cos(\alpha)|$$

The coordinates of the next pixel in the column vector is then given by:

$$(f(i) + \delta x, i + \delta y)$$

The coordinates of the pixel following that is determined by adding δx and δy to the previous coordinates accordingly, and so on, for each of the m pixels in the given column vector. The end-result is that this operation covers all the $m \cdot \sin(\alpha)$ pixels in the x -axis and all the $m \cdot \cos(\alpha)$ pixels in the y -axis, therefore mapping all of the m pixels in the specific column vector.

This process continues for all of the n column vectors in the image, until the entire image is rotated. In practice, each column vector is actually mapped **twice**. The first mapping happens from the i th column vector in I to the i th column vector in R . The second mapping happens from the i th column vector in I to the $(i+1)$ th column vector in R . Each of these $(i+1)$ th column vectors get overwritten in each iteration with a new column vector in I ; however, this gives a sufficiently accurate issuance of anti-aliasing without losing much quality.

The signs for δx and δy depend on whether, while mapping the column vectors in the rotated image, the values of the x -coordinate y -coordinate are decreasing or increasing. In the given example in figure 1, δx is negative, but δy is positive. There is a third parameter, defined as:

$$\delta i = \frac{1}{\delta x}$$

δi determines the rate at which the column vector is processed from one to the next, and its sign depends on the angle of rotation, α . The calculations for δx , δy , and δi only happen once per column vector in the image, so the entire image transformation is quite fast, overall.

B. Rotation Of An Image In The Row Major

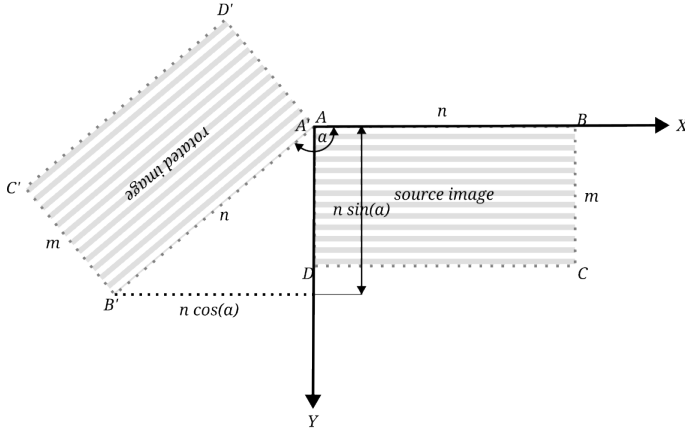


Fig. 2: Example demonstrating rotation of an image in the row major. Every row vector of pixels are rotated at once (Represented by the stripes in the image).

In the example image in figure 2, for the given angle of rotation, row vectors were rotated (represented by the stripes).

TABLE I: Notation

Symbol	Description
I	Original image
m, n	Height, width of I
R	Rotated image
m', n'	Height, width of R
α	Angle of rotation in radians
x'	Offset by which the x -coordinates of R need to be shifted
y'	Offset by which the y -coordinates of R need to be shifted
$f(\dots)$	Function defining the equation of the line from where the row or column vector starts [eq. 1] [eq. 2]
r_o	Upper limit of the function f
N	Number of elements (pixels) in the row or column vector f
flip	Boolean value that changes pixel processing order based on α
row_major	Boolean value that determines whether processing is to be done in the row or column major
negate	Boolean value, that if true, negates the indices used for indexing the pixels of the image

Note: The algorithm described in [Algorithm 1] follows *Python* style indexing. For example, if $a = [[1, 2, 3], [5, 6]]$, then $a[0][-1]$ represents the element 3.

All the calculations and logic is very similar to the one described for rotation in the column major [III-A]. Here, each vector to be rotated begins from the line $A'D'$, whose equation is given by:

$$f(i) = \left\lfloor \frac{i}{\tan(\alpha + \frac{\pi}{2})} \right\rfloor \forall \{i \in \mathbb{Z}, i \in [0, m \cdot \cos(\alpha))\} \quad (2)$$

Everything else stays the same, except the fact that now, each row vector is spread through a length of $n \cdot \cos(\alpha)$ in the x -axis and through a length of $n \cdot \sin(\alpha)$ in the y -axis, and accordingly, we have:

$$\delta x = |\cos(\alpha)|$$

$$\delta y = |\sin(\alpha)|$$

C. Implementation

Algorithm 1 Image Rotation Algorithm

```

1: procedure ROTATE(...) ▷ Performs image rotation on  $R$ 
2:   src_i = 0 ▷ Initialize internal control variables
3:   S = 1
4:   M = []
5:   if negate then
6:     S = -1
7:   end if
8:   if flip then
9:     M =  $[m_i]$  for  $i = 0, 1, \dots, N - 1$ ,  $| m_i = N - i$ 
10:  else
11:    M =  $[m_i]$  for  $i = 0, 1, \dots, N - 1$ ,  $| m_i = N$ 
12:  end if
13:  for  $i = 0$  to  $r_o$  do
14:    temp_x =  $f(i)$ 
15:    temp_y =  $i$ 
16:    for px in M do

```

```

17:     x = ⌊ temp_x ⌋
18:     y = ⌊ temp_y ⌋
19:     temp_i = ⌊ src_i ⌋
20:     if row_major then
21:         a = temp_i
22:         b = px
23:     else
24:         a = px
25:         b = temp_i
26:     end if
27:     ▷ Map rotated pixels to R
28:     R[ S * (y + y'), S * (x + x')] = I[a, b]
29:     R[ S * (y + y') + 1, S * (x + x')] = I[a, b]
30:     temp_x += δx
31:     temp_y += δy
32: end for
33: src_i += δi
34: end procedure

```

The algorithm described above [Algorithm 1] rotates image I to R .

R is Initialized as a blank image of width n' and height m' to fit R in the canvas, where:

$$m' = \lceil |m \cdot \cos(\alpha) + n \cdot \sin(\alpha)| \rceil$$

$$n' = \lceil |m \cdot \sin(\alpha) + n \cdot \cos(\alpha)| \rceil$$

The rotation function is then invoked, with each of the parameters updating depending on the value of α , as discussed above. The detailed values of each of the parameters are mentioned in [Table II].

Note: It has been observed, that for ranges of α which are vertically opposite each other (For example, angles in the range $(\frac{\pi}{4}, \frac{\pi}{2}]$ are vertically opposite angles in the range $(\frac{5\pi}{4}, \frac{3\pi}{2}]$, and so on), the only parameters of rotation that change are δi , negate , and flip . As such, the calculations have been generalized for such angles and they have been represented in [Table II], with a (') at the end of their names (shaded cells).

Algorithm 2 Driver Function

```

1: procedure MAIN(...)           ▷ Invokes [Algorithm 1]
2:   Determine the boolean value of row_major
3:   Determine  $f$ , based on the value of row_major
4:   Initialize a blank image  $R$  with dimensions  $(m', n')$ 
5:   Determine the parameters of rotation from [Table II]
6:   Generate  $R$  using [Algorithm 1]
7: end procedure

```

IV. EXPERIMENTAL RESULTS AND ANALYSIS

In this section, we perform an accuracy analysis of the reconstructed images using various error metrics. These metrics quantify the similarity between the original and the processed images, providing insight into the effectiveness of the image processing techniques. The following subsections describe each metric in detail, including their mathematical formulation and interpretation.

A. Peak Signal-to-Noise Ratio (PSNR)

PSNR is a widely used metric for evaluating the quality of reconstructed or compressed images. It quantifies the ratio between the maximum possible pixel value of the image and the mean squared error (MSE) between the original and the processed images. PSNR is expressed in decibels (dB), with higher values indicating better image quality.

$$\text{PSNR} = 20 \cdot \log_{10} \left(\frac{\text{MAX}_I}{\sqrt{\text{MSE}}} \right) \quad (3)$$

The Mean Squared Error (MSE) is calculated as:

$$\text{MSE} = \frac{1}{MN} \sum_{i=1}^M \sum_{j=1}^N [I(i, j) - K(i, j)]^2 \quad (4)$$

where MAX_I represents the maximum possible pixel value (e.g., 255 for 8-bit images), and $I(i, j)$ and $K(i, j)$ are the pixel values at position (i, j) in the original and processed images, respectively.

B. Correlation Coefficient (CC)

The Correlation Coefficient (CC) is a statistical measure that quantifies the similarity between two images by assessing the correlation between corresponding pixels. CC ranges from -1 to 1, with 1 indicating perfect positive correlation, -1 indicating perfect negative correlation, and 0 indicating no correlation.

$$\text{CC} = \frac{\sum_{i=1}^M \sum_{j=1}^N (I(i, j) - \bar{I}) (K(i, j) - \bar{K})}{\sqrt{\sum_{i=1}^M \sum_{j=1}^N (I(i, j) - \bar{I})^2 \cdot \sum_{i=1}^M \sum_{j=1}^N (K(i, j) - \bar{K})^2}} \quad (5)$$

Here, $\bar{I}$ and $\bar{K}$ represent the mean pixel values of the original and processed images, respectively.

C. Laplacian Mean Squared Error (LMSE)

Laplacian Mean Squared Error (LMSE) focuses on the high-frequency components of an image, particularly edges. By applying a Laplacian filter to both the original and processed images, LMSE calculates the error in edge details, which is particularly useful for detecting blurring or loss of fine details.

$$\text{LMSE} = \frac{1}{MN} \sum_{i=1}^M \sum_{j=1}^N [\text{Laplacian}(I(i, j)) - \text{Laplacian}(K(i, j))]^2 \quad (6)$$

Lower LMSE values indicate less error in edge preservation, thus implying higher image quality.

TABLE II: Rotation Parameters, based on α

Range of α	Rotation parameters											
	x'	y'	δx	δy	δi	r_o	N	negate	flip	$\delta i'$	negate'	flip'
$[0, \frac{\pi}{4}]$	$\lceil m \cdot \sin(\alpha) \rceil$	0	$ \cos(\alpha) $	$ \sin(\alpha) $	$1/\delta x$	$\lceil m \cdot \cos(\alpha) \rceil$	n	false	false	$1/\delta x$	true	false
$(\frac{\pi}{4}, \frac{\pi}{2}]$	$\lceil m \cdot \sin(\alpha) \rceil$	0	$- \sin(\alpha) $	$ \cos(\alpha) $	$-1/\delta x$	$\lceil n \cdot \sin(\alpha) \rceil$	m	false	false	$1/\delta x$	false	true
$(\frac{\pi}{2}, \frac{3\pi}{4}]$	m'	$\lceil m \cdot \cos(\alpha) \rceil$	$- \sin(\alpha) $	$- \cos(\alpha) $	$-1/\delta x$	$\lceil n \cdot \sin(\alpha) \rceil$	m	false	false	$1/\delta x$	false	true
$(\frac{3\pi}{4}, \pi]$	$\lceil n \cdot \cos(\alpha) \rceil$	0	$- \cos(\alpha) $	$ \sin(\alpha) $	$1/\delta x$	$\lceil m \cdot \cos(\alpha) \rceil$	n	false	false	$1/\delta x$	true	false

D. Relative Error (Re)

Relative Error (Re) quantifies the overall difference between the original and processed images, normalized by the magnitude of the original image's pixel values. This metric is useful for assessing the general accuracy of image reconstruction.

$$\text{Re} = \frac{\sum_{i=1}^M \sum_{j=1}^N |I(i, j) - K(i, j)|}{\sum_{i=1}^M \sum_{j=1}^N |I(i, j)|} \quad (7)$$

Lower values of Re indicate better similarity between the images.

E. Structural Similarity Index (SSIM)

The Structural Similarity Index (SSIM) measures the perceptual difference between two images. Unlike MSE-based metrics, SSIM considers changes in structural information, luminance, and contrast, making it more aligned with human visual perception. SSIM ranges from -1 to 1, with 1 indicating identical images.

$$\text{SSIM}(I, K) = \frac{(2\mu_I\mu_K + C_1)(2\sigma_{IK} + C_2)}{(\mu_I^2 + \mu_K^2 + C_1)(\sigma_I^2 + \sigma_K^2 + C_2)} \quad (8)$$

In this formula, μ_I and μ_K are the mean values, σ_I^2 and σ_K^2 are the variances, and σ_{IK} is the covariance of the original and processed images. The constants C_1 and C_2 are used to stabilize the formula when the denominator is close to zero, typically defined as:

$$C_1 = (k_1 L)^2, \quad C_2 = (k_2 L)^2 \quad (9)$$

where L is the dynamic range of the pixel values (e.g., 255 for 8-bit images), and k_1 and k_2 are small constants (e.g., $k_1 = 0.01$, $k_2 = 0.03$).

V. CONCLUSION

The conclusion goes here.

ACKNOWLEDGMENT

The authors would like to thank...

TABLE III: Error Analysis of Various Sample Images

Image	PSNR (dB)	CC	LMSE	Re	SSIM
Image 1	35.6	0.98	0.002	0.05	0.95
Image 2	33.2	0.96	0.003	0.07	0.92
Image 3	29.8	0.93	0.006	0.12	0.88
Image 4	40.1	0.99	0.001	0.03	0.97

REFERENCES

- [1] B. Gaster, L. Howes, D. R. Kaeli, P. Mistry, and D. Schaa, *Heterogeneous computing with openCL: revised openCL 1*. Newnes, 2012.
- [2] A. H. Ashtari, M. J. Nordin, and S. M. M. Kahaki, "Double line image rotation," *IEEE Transactions on Image Processing*, vol. 24, no. 11, pp. 3370–3385, 2015.
- [3] W. Park, G. Leibon, D. N. Rockmore, and G. S. Chirikjian, "Accurate image rotation using hermite expansions," *IEEE Transactions on Image Processing*, vol. 18, no. 9, pp. 1988–2003, 2009.
- [4] H.-C. Kwon, H.-J. Cho, and H.-Y. Kwon, "A study on high speed image rotation algorithm using cuda," *The Journal of the Institute of Internet, Broadcasting and Communication*, vol. 16, no. 5, pp. 1–6, 2016.
- [5] P. R. Evans, "Rotations and rotation matrices," *Acta Crystallographica Section D: Biological Crystallography*, vol. 57, no. 10, pp. 1355–1359, 2001.